

The Carvalho (2025) approximation estimates the wavelength of linear surface gravity waves directly from wave period and water depth, without iteration. The underlying wave dispersion relation is

$$\omega^2 = gk \tanh(kd), \quad \omega = \frac{2\pi}{T}, \quad k = \frac{2\pi}{L}.$$

Define the dimensionless parameter α and the deep-water wavelength L_0 as

$$\alpha = \frac{4\pi^2 d}{gT^2}, \quad L_0 = \frac{gT^2}{2\pi}.$$

The approximate wavelength is

$$L = L_0 \tanh[1.199315(\alpha^{1.047086}) \alpha^{0.499947}].$$

Use PERIOD for T in seconds and DEPTH for d in metres. Depth is the positive vertical distance from the still-water surface to the bed. Each function has the interface LWAVE(PERIOD, DEPTH) and returns L in metres, using the exact standard acceleration of gravity $g_n = 9.80665 \text{ m/s}^2$. Inputs must be finite and strictly positive.

For $\alpha \geq 20$, the functions return L_0 : the hyperbolic tangent in the approximation is indistinguishable from one in double precision. This avoids overflow when evaluating the nested exponential in deep water.

In Python, the function uses only the standard library:

```
import math

def LWAVE(PERIOD: float, DEPTH: float) -> float:
    """Return wavelength [m] from period [s] and depth [m]."""
    if (not math.isfinite(PERIOD) or not math.isfinite(DEPTH)
        or PERIOD <= 0.0 or DEPTH <= 0.0):
        raise ValueError("PERIOD and DEPTH must be finite and > 0.")

    g = 9.80665
    L0 = g * PERIOD**2 / (2.0 * math.pi)
    alpha = 4.0 * math.pi**2 * DEPTH / (g * PERIOD**2)

    if alpha >= 20.0:
        return L0

    x = 1.199315**(alpha**1.047086) * alpha**0.499947
    return L0 * math.tanh(x)
```

In Java 8 or later, save this class as WaveDispersion.java and call WaveDispersion.LWAVE(PERIOD, DEPTH):

```
public final class WaveDispersion {
    public static double LWAVE(double PERIOD, double DEPTH) {
        if (!Double.isFinite(PERIOD) || !Double.isFinite(DEPTH)
            || PERIOD <= 0.0 || DEPTH <= 0.0) {
            throw new IllegalArgumentException(
                "PERIOD and DEPTH must be finite and > 0.");
        }

        final double g = 9.80665;
        final double pi = Math.PI;
        final double L0 = g * PERIOD * PERIOD / (2.0 * pi);
        final double alpha = 4.0 * pi * pi * DEPTH
            / (g * PERIOD * PERIOD);

        if (alpha >= 20.0) {
            return L0;
        }

        final double x = Math.pow(1.199315,
            Math.pow(alpha, 1.047086))
            * Math.pow(alpha, 0.499947);
        return L0 * Math.tanh(x);
    }
}
```

The equivalent C++ function requires C++11 or later:

```
#include <cmath>
#include <stdexcept>

double LWAVE(double PERIOD, double DEPTH)
{
    if (!std::isfinite(PERIOD) || !std::isfinite(DEPTH)
        || PERIOD <= 0.0 || DEPTH <= 0.0) {
        throw std::invalid_argument(
            "PERIOD and DEPTH must be finite and > 0.");
    }

    const double g = 9.80665;
    const double pi = std::acos(-1.0);
    const double L0 = g * PERIOD * PERIOD / (2.0 * pi);
    const double alpha = 4.0 * pi * pi * DEPTH
        / (g * PERIOD * PERIOD);
```

```

    if (alpha >= 20.0) {
        return L0;
    }

    const double x = std::pow(1.199315, std::pow(alpha, 1.047086))
        * std::pow(alpha, 0.499947);
    return L0 * std::tanh(x);
}

```

In Fortran 2008 or later, save the module as wave_dispersion.f90. Import it with use wave_dispersion, only: LWAVE and pass real(real64) arguments. For example, after importing real64 from iso_fortran_env, use LWAVE(9.0_real64, 5.0_real64):

```

module wave_dispersion
    use, intrinsic :: iso_fortran_env, only: real64
    use, intrinsic :: ieee_arithmetic, only: ieee_is_finite
    implicit none
contains
    function LWAVE(PERIOD, DEPTH) result(L)
        real(real64), intent(in) :: PERIOD, DEPTH
        real(real64), parameter :: g = 9.80665_real64
        real(real64), parameter :: pi = acos(-1.0_real64)
        real(real64) :: L, L0, alpha, x

        if (.not. ieee_is_finite(PERIOD) .or. &
            .not. ieee_is_finite(DEPTH) .or. &
            PERIOD <= 0.0_real64 .or. DEPTH <= 0.0_real64) then
            error stop "PERIOD and DEPTH must be finite and > 0."
        end if

        L0 = g * PERIOD**2 / (2.0_real64 * pi)
        alpha = 4.0_real64 * pi**2 * DEPTH / (g * PERIOD**2)

        if (alpha >= 20.0_real64) then
            L = L0
            return
        end if

        x = 1.199315_real64**(alpha**1.047086_real64) &
            * alpha**0.499947_real64
        L = L0 * tanh(x)
    end function LWAVE
end module wave_dispersion

```

In MATLAB, save the following function as LWAVE.m. It accepts scalar numeric inputs and performs the calculation in double precision:

```
function L = LWAVE(PERIOD, DEPTH)
    validateattributes(PERIOD, {'numeric'}, ...
        {'real', 'scalar', 'finite', 'positive'}, ...
        mfilename, 'PERIOD', 1);
    validateattributes(DEPTH, {'numeric'}, ...
        {'real', 'scalar', 'finite', 'positive'}, ...
        mfilename, 'DEPTH', 2);

    PERIOD = double(PERIOD);
    DEPTH = double(DEPTH);
    g = 9.80665;
    L0 = g * PERIOD^2 / (2 * pi);
    alpha = 4 * pi^2 * DEPTH / (g * PERIOD^2);

    if alpha >= 20
        L = L0;
        return
    end

    x = 1.199315^(alpha^1.047086) * alpha^0.499947;
    L = L0 * tanh(x);
end
```

For Excel VBA, paste the following function into a standard module. It returns a numeric result, or #NUM! for non-positive inputs or an arithmetic error. WorksheetFunction.Tanh supplies the hyperbolic tangent:

Option Explicit

```
Public Function LWAVE(ByVal PERIOD As Double, _
    ByVal DEPTH As Double) As Variant
    Const g As Double = 9.80665
    Const PI As Double = 3.141592653589793
    Dim L0 As Double
    Dim alpha As Double
    Dim x As Double

    On Error GoTo InvalidInput
    If PERIOD <= 0# Or DEPTH <= 0# Then GoTo InvalidInput

    L0 = g * PERIOD ^ 2 / (2# * PI)
    alpha = 4# * PI ^ 2 * DEPTH / (g * PERIOD ^ 2)

    If alpha >= 20# Then
```

```

    LWAVE = L0
Else
    x = 1.199315 ^ (alpha ^ 1.047086) _
        * alpha ^ 0.499947
    LWAVE = L0 * Application.WorksheetFunction.Tanh(x)
End If
Exit Function

```

InvalidInput:

```

    LWAVE = CVErr(xlErrNum)
End Function

```

For example, with $T = 9$ s and $d = 5$ m, LWAVE(9.0, 5.0) gives approximately **60.399342 m**. In Excel, enter =LWAVE(A1,B1) with 9 in A1 and 5 in B1; use a semicolon instead of a comma if required by the regional settings. Save the workbook as .xlsm to retain the VBA function.